SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Transformer Mechanics Worksheet

Course Code:	CSA6502	**Course Title:**	Generative AI and Large Scale Model
CO Assessed:	CO1-Imitation (BL3)	**Assessment:**	Assessment Tool 3– **Transformer Mechanics Worksheet**
Weightage:	10% of CIA	**Total Marks:**	20
CO1 Statement:	Explain the fundamentals of Artificial Intelligence, Generative AI, Foundation Models, Transformer architecture, tokenization, embeddings, and Large Language Models.		
Student Name:	S. Muskan	**Reg. No.:**	192472182
Section / Batch:	Slot A	**Date:**	21-07-2026

Code of Conduct

I, S. Muskan(192472182), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S.Muskan

Instructions

- This assessment contains 4 set of questions.
- Each student assigned with one set of questions .
- Each question carries 5 marks. Total: 20 Marks.
- Ensure clarity, structure, and logical flow in diagrams.
- Duration: 20 Minutes.

Annexure A Transformer Mechanics Worksheet

Rubrics for Calculation

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Lev
Technical / Concept	30%	Accurately explains tokenization, embeddings, and self-attention mechanics using correct terminology	Explains most mechanics correctly with minor gaps	Partial or superficial understanding of the mechanics	Maj mis cor
Application	25%	Correctly traces a worked example (e.g. computing attention relevance) step-by-step	Traces most steps correctly with minor errors	Significant errors when applying mechanics to the worked example	Can mec wor all
Quality	20%	Clear, logical, mathematically sound reasoning throughout	Mostly sound reasoning with minor errors	Reasoning has notable gaps or inconsistencies	Rea disc larg
Presentation	15%	Neat, well-labeled diagrams/working, easy to follow	Reasonably clear presentation of working	Presentation is unclear in places	Diffi pre wor
Documentation / Communication	10%	Shows all work with clear justification for every step	Shows most work with adequate justification	Minimal work shown	No ans unju

Question 1: Tokenization

Title

Sub-word Tokenization of the Word "DISAGREEMENT"

Aim

To understand how sub-word tokenization works in Transformer models and why it is preferred over whole-word tokenization.

Problem Statement

A Transformer model uses sub-word tokenization to process uncommon or unseen words efficiently. Instead of treating an entire unfamiliar word as a single token, it breaks the word into meaningful sub-word units that can be understood using previously learned vocabulary.

The task is to:

- Split the word **"DISAGREEMENT"** into three meaningful sub-word tokens.
- Justify the chosen split.
- Explain why sub-word tokenization helps when the complete word has never appeared during training.

Tasks

(a) Split "DISAGREEMENT" into 3 reasonable sub-word pieces and justify your split.

Answer:

One possible split is:

DIS | AGREE | MENT

Justification:

- **DIS** is a common prefix indicating negation.
- **AGREE** is the root word.
- **MENT** is a common suffix used to form nouns.

Since these sub-words frequently appear in many other words, the tokenizer can reuse them to understand the complete word.

(b) Explain why sub-word tokenization helps.

Answer:

Sub-word tokenization allows the model to understand new or rare words by combining familiar prefixes, roots, and suffixes. Even if the complete word **"DISAGREEMENT"** was never seen during training, the model already knows the meanings of **DIS**, **AGREE**, and **MENT**, enabling it to infer the meaning of the entire word.

Program:

```
word = input("Enter a word: ").upper()

if word == "DISAGREEMENT":
    tokens = ["DIS", "AGREE", "MENT"]
else:
    tokens = [word[:3], word[3:7], word[7:]]

print("\nSub-word Tokens:")
for token in tokens:
    print(token)
```

Output:

```
Enter a word: DISAGREEMENT

Sub-word Tokens:
DIS
AGREE
MENT


** Process exited - Return Code: 0 **
```

Question 2: Embeddings

Title

Understanding Word Embeddings and Cosine Similarity

Aim

To understand semantic similarity between words using embeddings and compare embeddings with one-hot vectors.

Problem Statement

Transformer models represent words as dense numerical vectors called embeddings. Similar words are placed close together in vector space, while unrelated words are placed farther apart.

The task is to:

- Compare the similarity of two word pairs.
- Explain the concept of embeddings.
- Differentiate embeddings from one-hot vectors.

Tasks

(a) Which pair has higher cosine similarity and why?

Answer:

The pair **"dog"** and **"puppy"** will have a **higher cosine similarity** because both words are semantically related. A puppy is a young dog, so their embeddings are located close to each other in vector space.

The pair **"dog"** and **"truck"** are unrelated, so their cosine similarity will be much lower.

(b) What is an embedding? How is it different from a one-hot vector?

Answer:

An embedding is a dense numerical representation of a word that captures its meaning and relationship with other words. Similar words have similar embeddings.

A one-hot vector simply identifies a word's position in the vocabulary and does not store any semantic meaning. Embeddings, on the other hand, encode contextual and semantic information.

Program:

```
import numpy as np

dog = np.array([0.8, 0.7, 0.2])
puppy = np.array([0.82, 0.68, 0.25])
truck = np.array([0.1, 0.2, 0.95])

def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print("Dog vs Puppy :", cosine_similarity(dog, puppy))
print("Dog vs Truck :", cosine_similarity(dog, truck))
```

Output:

```
Dog vs Puppy : 0.9986724157294657
Dog vs Truck : 0.3883814096272585


** Process exited - Return Code: 0 **
```

Question 3: Self-Attention Worked Example

Title

Resolving Word References Using Self-Attention

Aim

To understand how self-attention helps Transformer models identify relationships between words in a sentence.

Problem Statement

In natural language, pronouns often refer to earlier words in a sentence. Transformer models use self-attention to determine which previous words are most relevant instead of relying only on word order.

Sentence:

"The city council refused the protesters a permit because they feared violence."

The task is to:

- Identify what the word **"they"** refers to.
- Explain why self-attention is necessary.

Tasks

(a) "They" should attend most strongly to which word?

Answer:

City council

(b) Explain why self-attention is needed.

Answer:

Self-attention enables the model to consider the relationships between all words in the sentence simultaneously. It understands that the phrase **"feared violence"** is more logically associated with the **city council**, allowing it to correctly resolve the pronoun **"they"**. Simple word-order rules cannot reliably determine such relationships in complex sentences.

Program:

```
sentence = ["The","city","council","refused","the",
            "protesters","a","permit","because",
            "they","feared","violence"]

attention = {
    "they":{
        "city council":0.90,
        "protesters":0.10
    }
}

print("Sentence:")
print(" ".join(sentence))

print("\nAttention Scores for 'they'")
for word, score in attention["they"].items():
    print(word, ":", score)

print("\nPrediction:")
print("'they' refers to -> city council")
```

Output:

```
Sentence:
The city council refused the protesters a permit because they feared violence

Attention Scores for 'they'
city council : 0.9
protesters : 0.1

Prediction:
'they' refers to -> city council


** Process exited - Return Code: 0 **
```

Question 4: Query, Key, Value

Title

Understanding Query, Key, and Value in Self-Attention

Aim

To understand the roles of Query, Key, and Value vectors in the self-attention mechanism of Transformer models.

Problem Statement

Self-attention enables each word in a sentence to determine which other words are most relevant while processing information. This is achieved using Query, Key, and Value vectors.

The task is to explain:

- The purpose of Query, Key, and Value vectors.
- How they work together to compute attention scores.

Tasks

Answer:

- **Query (Q):** Represents the information that the current word is searching for from other words.
- **Key (K):** Represents the characteristics of each word that can be matched against the Query.
- **Value (V):** Contains the actual information that will be passed forward if the corresponding Key is considered important.

The attention score is calculated by comparing the **Query** of one word with the **Keys** of all other words. Higher similarity results in a higher attention score, and the corresponding **Values** contribute more to the final representation of the word.

Program:

```
import numpy as np

Q = np.array([1, 0])
K = np.array([
    [1, 0],
    [0, 1],
    [1, 1]
])
V = np.array([
    [10, 5],
    [4, 8],
    [7, 6]
])
scores = np.dot(K, Q)
print("Attention Scores:")
print(scores)
weights = scores / np.sum(scores)
print("\nAttention Weights:")
print(weights)
output = np.dot(weights, V)
print("\nFinal Attention Output:")
print(output)
```

Output:

```
Attention Scores:
[1 0 1]

Attention Weights:
[0.5 0.  0.5]

Final Attention Output:
[8.5 5.5]


** Process exited - Return Code: 0 **
```